데이터 분석을 위한 pandas 1

학습목표

1) pandas 의 Series 와 DataFrame 을 만든다

Series · DataFrame, 인덱스와 열, NumPy 배열과의 관계

2) 파일을 읽고 저장한다

Colab · Google Drive 경로, CSV · Excel, 인코딩 · 경로 오류

3) 표의 구조와 자료형을 확인하고 바꾼다

head · info · describe, 결측 · 중복 · value_counts, 날짜형(.dt), 문자열 숫자 정리

4) 필요한 행과 열을 고르고 조건으로 거른다

loc · iloc, & · |, isin · between, query, str.contains, nlargest, axis

pandas 1의 흐름

1부

Series·DataFrame · 파일

표 만들기, Drive 연결, CSV·Excel 읽기·저장

2부

구조 확인 · 자료형

head·info·describe, isna, value_counts, to_datetime·.dt, astype

3부

선택 · 조건 조회

loc·iloc, 조건, isin·between, query, str.contains, axis

Series와 DataFrame

NumPy 는 배열, pandas 는 데이터프레임

NumPy → 배열을 다루는 도구

```
a = np.array([[240, 300],
              [180, 270]])
a[0, 1]      # 300  (0번 행, 1번 열)
```

pandas → 데이터프레임을 다루는 도구

```
df = pd.DataFrame({"라인": ["A라인", "B라인"],
                   "생산량": [240, 180]})
df.loc[0, "생산량"]   # 240  (0번 행, 생산량 열)
```

인덱스(index): 행의 이름. 따로 정하지 않으면 0부터 번호가 붙는다.
공식 문서는 label이라 부른다
columns: 열 이름 목록 · 1차원은 Series, 2차원은 DataFrame

pandas 10분 입문: https://pandas.pydata.org/docs/user_guide/10min.html · NumPy 기초: https://numpy.org/doc/stable/user/absolute_beginners.html

Series

Series = 인덱스(행 이름)가 붙은 1차원 배열

```
output = pd.Series([240, 300, 180],
    index=["A라인", "B라인", "C라인"])
print(output)
# A라인    240
# B라인    300
# C라인    180
# dtype: int64

print(output.loc["B라인"])  # 300  이름으로 찾기
print(output.iloc[0])  # 240  위치로 찾기
print(output.mean())   # 240.0
```

이름이 붙은 값 목록



Series는 인덱스(행 이름)가 붙은 1차원 배열이다.
위치 번호 대신 이름으로도 값을 찾을 수 있다.
DataFrame의 열 하나가 Series다.

DataFrame

DataFrame = 여러 Series를 묶은 표

```
df = pd.DataFrame({
    "라인": ["A라인", "B라인", "C라인"],
    "생산량": [240, 300, 180],
})  # {열이름: 값 목록} 딕셔너리
print(df)
#    라인   생산량
# 0  A라인  240
# 1  B라인  300
# 2  C라인  180
```

엑셀 시트 한 장과 같은 구조

	라인	생산량
0	A라인	240
1	B라인	300
2	C라인	180

■ 행(row) = 기록 한 건

□ 열(column) = 항목 하나

행(row)은 기록 한 건, 열(column)은 항목 하나인 2차원 표다. 엑셀 시트 한 장과 같은 구조이며, 각 열은 서로 다른 자료형을 가질 수 있다.

Series와 DataFrame

구분	Series	DataFrame
구조	1차원, 인덱스 이름이 붙은 배열	2차원, 행·열 이름이 있는 표 열마다 자료형이 다를 수 있다
생성(constructor)	pd.Series(리스트, index=인덱스 이름)	pd.DataFrame({열 이름: 리스트})
이름	index 하나	index(행) + columns(열)
선택(indexing)	이름으로 찾기: s.loc["A라인"] 위치로 찾기: s.iloc[0]	열 하나: df["열"] 이름으로 찾기: df.loc[행 이름, 열 이름] 위치로 찾기: df.iloc[행 번호, 열 번호]
요약 통계 (descriptive statistics)	s.mean(), s.describe()	df["열"].mean(), df.describe()
파이썬 구조	리스트 + 인덱스 이름	딕셔너리 {열: 리스트}

Series·DataFrame 소개: https://pandas.pydata.org/docs/user_guide/dsintro.html · 요약 통계: https://pandas.pydata.org/docs/user_guide/basics.html#descriptive-statistics

Series와 DataFrame

DataFrame (2차원 표)

인덱스	라인	생산량
0	A라인	240
1	B라인	300
2	C라인	180

`df["생산량"]`

→

Series (1차원 한 열)

인덱스	생산량
0	240
1	300
2	180

DataFrame에서 열 하나를 꺼내면 Series

행(row)은 기록 한 건, 열(column)은 변수(측정한 항목) 하나
Series = 값 + 인덱스 이름, DataFrame = 여러 Series를 묶은 표

Series 의 구조: 이름 · 인덱스 · 값

Series 는 이름(name) · 인덱스(index) · 값(values)

	name: 생산량
A라인	240
B라인	300
C라인	180

인덱스 | 값 (1차원 배열)

```
output = pd.Series([240, 300, 180],
    index=["A라인", "B라인", "C라인"], name="생산량")
output.index        # A라인, B라인, C라인
output.to_numpy()   # [240 300 180]
output.name         # '생산량'
```

Series.to_numpy: https://pandas.pydata.org/docs/reference/api/pandas.Series.to_numpy.html

Series 두 개가 모이면 DataFrame

concat(연결): 여러 Series·DataFrame 을 이어 붙인다. axis=1 이면 옆으로(열로) 붙어 DataFrame 이 된다

	생산량
A라인	240
B라인	300
C라인	180

+

	불량수
A라인	5
B라인	5
C라인	5

=

	생산량	불량수
A라인	240	5
B라인	300	5
C라인	180	5

DataFrame

```
df = pd.concat([output, defect], axis=1)   # Series 두 개를 옆으로(axis=1) 붙이기
df["생산량"]                                # 열 하나를 꺼내면 다시 Series
```

pd.concat: https://pandas.pydata.org/docs/reference/api/pandas.concat.html

NumPy와 pandas · 결측값이 있는 평균

```
values = [150, np.nan, 250]
print(np.array(values).mean())                # nan
print(np.nanmean(values))                     # 200.0
print(pd.Series(values).mean())               # 200.0
print(pd.Series(values).mean(skipna=False))  # nan
```

150	결측	250

→ (150 + 250) ÷ 2건 = 200.0

코드	실행	결과
`np.array(values).mean()`	결측값을 그대로 둔다 → 계산 전체가 nan	nan
`np.nanmean(values)`	결측값을 빼고 계산한다	200.0
`pd.Series(values).mean()`	결측값을 빼고 계산한다 (기본값 skipna=True)	200.0
`pd.Series(values).mean(skipna=False)`	결측값을 그대로 둔다 → nan	nan

np.nanmean: https://numpy.org/doc/stable/reference/generated/numpy.nanmean.html · Series.mean: https://pandas.pydata.org/docs/reference/api/pandas.Series.mean.html

Colab에서 파일 다루기

Colab에서 파일 다루기

방법 A: Colab 임시 폴더

현재 작업 폴더(/content)에 저장한다. 빠르지만 런타임이 끊기면 사라진다. files.download() 로 내 컴퓨터에 받아 둔다.

```
df.to_csv("sales.csv", index=False)    # 저장 (index=False: 행 번호는 저장하지 않는다)
df = pd.read_csv("sales.csv")          # 다시 읽기
from google.colab import files         # Colab 다운로드 기능
files.download("sales.csv")            # 내 컴퓨터로 받기
```

방법 B: Google Drive 연결

drive.mount() 후 Drive 안의 폴더에 저장한다. 런타임이 끊겨도 남는다.

```
from google.colab import drive
drive.mount("/content/drive")
csv_path = "/content/drive/MyDrive/HD-AI/data/sales.csv"
```

read_csv: https://pandas.pydata.org/docs/reference/api/pandas.read_csv.html · to_csv: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.to_csv.html

파일을 읽을 때 자주 만나는 오류

오류	원인	확인·해결
`FileNotFoundError`	경로나 파일명이 틀렸다, Drive 가 연결되지 않았다	os.path.exists(경로) 로 확인, os.listdir(폴더) 로 파일명 확인
`UnicodeDecodeError`	파일의 한글 인코딩과 읽는 방식이 다르다	encoding="cp949" 또는 "utf-8-sig" 로 바꿔 읽는다

read_excel(): https://pandas.pydata.org/docs/reference/api/pandas.read_excel.html

데이터 구조 확인

데이터 구조 확인

1) `head(n) / tail(n)`

처음·마지막 n행

확인하는 이유: 열 구성, 한글 깨짐, 파일 끝의 합계 행

2) `sample(n, random_state=42)`

무작위 n행

확인하는 이유: 정렬된 데이터의 앞부분만 보고 착각하는 것 방지. random_state 로 결과 고정

3) `shape`

(행 수, 열 수) 튜플

확인하는 이유: 데이터가 모두 몇 건인지 파악.

4) `columns / dtypes`

열 이름 / 열별 자료형

확인하는 이유: 숫자가 문자열로 들어오지 않았는지

5) `info()`

행 수·열·결측 아닌 개수·자료형 한 번에

6) `isna().sum()`

열별 결측값 개수

확인하는 이유: 어느 열이 얼마나 비어 있는지 파악

DataFrame.info: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.info.html · isna: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.isna.html · DataFrame.sample(random_state): https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.sample.html

데이터 구조 확인

속성(shape, columns, dtypes)은 괄호가 없고, 메서드(head(), info(), isna())는 괄호가 있다

columns: 열 이름 줄

head(2)
앞 2행

	작업일	라인	금액
0	2026-01-05	A라인	1800000
1	2026-01-12	B라인	650000
2	2026-01-20	A라인	240000
3	2026-02-03	C라인	1200000
4	2026-02-10	B라인	360000

tail(2)
뒤 2행

shape → (5, 3)
5행 3열

dtypes → 열마다 자료형

참고: random_state란

random_state는 난수 생성의 시드(seed)를 고정하는 매개변수다. 컴퓨터의 랜덤값은 실제로는 시드값에서 출발해 정해진 규칙으로 만들어지는 의사 난수(pseudo-random)이므로, 같은 시드를 지정하면 실행할 때마다 같은 행이 뽑힌다. 결과를 재현할 수 있어 표준적으로 사용한다. 42라는 숫자 자체는 의미가 없는 관례다.

결측 비율 isna().mean() 과 중복 확인 duplicated()

코드	하는 일	결과 읽기
`df.isna().sum()`	열별 빈 칸 개수	True(1) 을 더한다
`df.isna().mean() * 100`	열별 빈 칸 비율(%)	True(1)·False(0) 의 평균 = 비율
`df.duplicated().sum()`	완전히 같은 행이 몇 개 더 있나	0 이면 중복 없음
`df.drop_duplicates()`	중복 행을 지우고 첫 행만 남긴다	원본은 그대로. 다시 대입한다
`df.drop_duplicates("열", keep="last")`	해당 열 값이 같으면 마지막 행만	중복된 데이터 중 현재 행 순서에서 마지막으로 나타난 행을 유지
`df["열"].nunique()`	해당열의 서로 다른 값의 가짓수	행 수와 비교해 식별자가 고유한지 확인

duplicated(): https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.duplicated.html · nunique(): https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.nunique.html

결측 처리: fillna() · dropna()

기능	코드	결과 (아래 예시 데이터)
값 하나로 채우기	`df["수량"].fillna(0)`	W002 수량 NaN → 0.0
중앙값으로 채우기	`df["수량"].fillna(df["수량"].median())`	W002 수량 NaN → 4.0 (중앙값)
열마다 다르게 채우기	`df.fillna({"수량": 0, "라인": "미입력"})`	수량 NaN → 0.0 · 라인 NaN → "미입력"
결측 있는 행 지우기	`df.dropna()`	하나라도 NaN 이면 삭제 → W001 만 남음
특정 열 기준으로 지우기	`df.dropna(subset=["금액"])`	금액이 NaN 인 W004 만 삭제
모두 비어 있는 행만	`df.dropna(how="all")`	모든 값이 NaN 인 행만 삭제 → 4행 그대로

예시 데이터 (결측이 있는 4행)

작업번호	라인	수량	금액
W001	A라인	4.0	1800000.0
W002	B라인	NaN	650000.0
W003	NaN	12.0	240000.0
W004	C라인	3.0	NaN

결과를 다시 담아야 바뀐다

```
df["수량"] = df["수량"].fillna(0)
df = df.dropna(subset=["금액"])
```

fillna(): https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.fillna.html · dropna(): https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.dropna.html

describe(): 수치형(int · float) 열의 요약 통계

행	뜻	예: 5일 생산량 [320, 350, 380, 410, 440]
`count`	비어 있지 않은 값의 개수	5
`mean`	평균	380.0
`std`	표준편차	47.43
`min / max`	최솟값 / 최댓값	320 / 440
`25%`	작은 순으로 늘어놓았을 때 아래에서 1/4 지점의 값 (제1사분위수)	350
`50%`	한가운데 값 = 중앙값 (제2사분위수)	380
`75%`	아래에서 3/4 지점의 값 (제3사분위수)	410

describe(): https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.describe.html

describe():문자형(str)열의 요약 통계

행	뜻	예: 라인 열 [A라인, B라인, A라인, C라인, B라인, A라인]
count	비어 있지 않은 값의 개수	6
unique	서로 다른 값의 종류 수	3
top	가장 많이 나온 값	A라인
freq	top 이 나온 횟수	3

```
df.describe(include="object")    # 글자 열만 요약
df.describe()                    # 숫자 열이 하나도 없으면 이것만으로도 글자 열 요약이 나온다
df.describe(include="all")       # 숫자·글자 열 전부. 해당 없는 칸은 NaN
```

describe(): https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.describe.html

자주 쓰는 요약 통계 메서드

항목	수치형 (int · float)	범주형 (문자 · category)
개수	count() 비어 있지 않은 수	nunique() 종류 수 · unique() 종류 목록
대표값	mean() 평균 · median() 중앙값	mode()[0] 가장 많은 값
값별 개수	—	value_counts() 개수 · value_counts(normalize=True) 비율
최댓값 · 최솟값	max() · min()	value_counts().max() 가장 많은 값의 개수
위치(인덱스)	idxmax() · idxmin() 최댓값·최솟값의 인덱스	value_counts().idxmax() 가장 많은 값
조건에 맞는 개수	(df["금액"] >= 1000000).sum() True 의 개수	(df["라인"] == "A라인").sum()

Series 메서드(요약 통계): https://pandas.pydata.org/docs/reference/series.html#computations-descriptive-stats

자료형 확인과 변환

자료형 확인과 변환

저장 자료형 (pandas)	담는 값	통계에서의 역할
`int64`	수량, 횟수	수치형 (계산 가능)
`float64`	금액, 온도, 비율, 결측값이 섞인 정수	수치형
`object / str`	이름, 코드, 판정	범주형 또는 식별자
`datetime64`	작업일, 측정일시	시간 변수 (정렬·기간 계산)
`bool`	True / False	이진 변수 (평균 = 비율)

to_datetime: https://pandas.pydata.org/docs/reference/api/pandas.to_datetime.html · astype: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.astype.html

형변환 메서드

CSV 로 읽은 날짜·금액은 글자로 들어오는 일이 많다. 글자 상태에서는 날짜 비교·.dt·계산이 안 되므로 먼저 자료형을 바꾼다.

바꾸는 방향	메서드	예	주의
문자 → 날짜	`pd.to_datetime(열, errors="coerce")`	"2026-03-19" → datetime64	못 바꾸는 값은 NaT. 바꾼 뒤에야 .dt.year · .dt.month · 기간 조회가 된다
문자 → 숫자	`pd.to_numeric(열, errors="coerce")`	"250" → 250 (NaN 이 섞이면 250.0)	못 바꾸는 값은 NaN. isna().sum() 으로 몇 개인지 확인
쉼표·단위 지우기	`.str.replace(",", "")` `.str.replace("원", "")`	"2,450,000" → "2450000"	지운 뒤 astype(int). 바로 astype 하면 ValueError
실수 → 정수	`astype(int)`	3.0 → 3	결측값이 있으면 일반 정수형 변환 시 오류가 발생
문자 → 범주형	`astype("category")`	"A라인" → "A라인" (자료형: category)	같은 글자가 반복되는 열(라인·등급)에 쓴다.
확인	`dtypes` `info()`	object / str · int64 · datetime64	괄호 없는 것은 속성, 있는 것은 메서드

.dt 는 날짜형 전용, .str 은 문자열 전용 접근자다.

to_datetime: https://pandas.pydata.org/docs/reference/api/pandas.to_datetime.html · to_numeric: https://pandas.pydata.org/docs/reference/api/pandas.to_numeric.html

문자열 메서드: .str

기능	pandas (열 전체)	예 (품목명 · 작업일)	파이썬 (값 하나)
포함 여부	`.str.contains("밸브")`	"볼밸브" → True	`"밸브" in s`
바꾸기	`.str.replace("DC", "직류")`	"DC모터" → "직류모터"	`s.replace("DC", "직류")`
시작 글자	`.str.startswith("DC")`	"DC모터" → True	`s.startswith("DC")`
소문자 · 대문자	`.str.lower()` · `.str.upper()`	"DC모터" → "dc모터"	`s.lower()` · `s.upper()`
나누기	`.str.split("-")`	"2026-01-05" → ["2026", "01", "05"]	`s.split("-")`
개수 세기	`.str.count("-")`	"2026-01-05" → 2	`s.count("-")`
길이	`.str.len()`	"DC모터" → 4	`len(s)`
앞 글자 자르기	`.str[:4]`	"2026-01-05" → "2026"	`s[:4]`

Working with text data: https://pandas.pydata.org/docs/user_guide/text.html · 파이썬 문자열 메서드: https://docs.python.org/ko/3/library/stdtypes.html#string-methods

날짜 메서드: .dt

기능	코드	예 (작업일)	주의
형식 지정 변환	`pd.to_datetime(열, format="%Y-%m-%d")`	"2026-01-05" → 2026-01-05	적힌 모양대로 format 을 쓴다
연 · 월 · 일	`.dt.year · .dt.month · .dt.day`	2026 · 1 · 5	괄호 없이 쓰는 속성이다
요일	`.dt.dayofweek`	0 (월요일)	월요일 0 ~ 일요일 6 요일 이름은 .dt.day_name()
월 단위로 묶기	`.dt.to_period("M")`	2026-01	groupby 기준으로 쓰면 월별 집계
날짜끼리 빼기	`df["작업일"] - df["작업일"].min()`	2026-01-12 → 7 days	결과는 기간(Timedelta)
기간을 숫자로	`.dt.days · .dt.total_seconds()`	7 · 604800.0	분 단위는 total_seconds() / 60
날짜 비교	`df[df["작업일"] >= "2026-04-01"]`	W011 ~ W015 (5행)	날짜형 열은 날짜 문자열과 바로 비교된다

형식 기호: %Y 연도(4자리) · %m 월 · %d 일 · %H 시 · %M 분 · %S 초 (파이썬 datetime 과 같음)

Time series / date functionality: https://pandas.pydata.org/docs/user_guide/timeseries.html · 형식 기호(파이썬): https://docs.python.org/ko/3/library/datetime.html#format-codes